

DOLA Multichain Hack Analysis



@TrophyCase Ok great, you're partially responsible. But you feel like your contribution to resolving this issue is commensurate with your responsibility?

Eagles 05.09.2023, 02:02

I'd like to come back to this. I've probably put 30-40 hours of work into this situation, broken down roughly as follows:

- 5-6 hours or so of on-chain sleuthing, reading MC docs, Inverse docs, verifying for myself and others in the initial hours after the hack what exactly had happened on chain, the structure of the MC contracts on BSC and on Arbitrum and that despite the fact that the Inverse DAO had full ownership of the contracts on BSC, nothing could be done because the anyDOLA contract on Arbitrum being ultimately owned by the Multichain "MPC" address.
- 2-3 hours developing a proposal wherein an insurance-based approach could be taken to re-issue DOLA if it hadn't been moved
- 3-4 hours verifying onchain that various other proposals submitted in here were indeed intractable during the week or so after the hack
- 15-20 hours of work fully reading Arbitrum's docs on governance and protocol upgrades, culminating in the proposal that I made which would result in Arbitrum upgrading their proxy contracts associated with bridging. As part of this, I spun up a foundry project which simulates all of this by impersonating the Arbitrum governance and upgrading their contracts on a fork. I had to interact with their bridge contracts across two forked chains, talk to ArbSys, and write quite a few smart contracts (mostly in the form of foundry scripts) to get an MVP working, after which I could be pretty confident in my hypothesis that this could work as a way forward. Despite the fact that I'd done all this work, the team continued to insist to me that it wasn't possible until I tagged someone directly technical and told them pretty strongly to **go and read the docs**.

Despite all the work I've put in, I'd be surprised if anyone on the team has done even 10% of this in terms of number of hours. I obviously haven't counted all the hours I've spent in here advocating on behalf of other DOLA holders.

I'm not a great fan of wasting effort on things that quite clearly have no hope of working, so that's why at this point, I'd like to see a little proactivity from the team, because there's absolutely no chance of getting Arbitrum to take any action without backing from the team. I don't want to pour in yet more effort when the more I give, I'm actually seeing the opposite results to what I'm trying to achieve - team is digging in deeper, and insisting that they won't do any work on this whatsoever. (edited)

👍 9 ❤️ 6 🗨️



Oxpri 26.07.2023, 23:49

Here is my write up about potentially being able to recover the 1.9m DOLA that is stored on multichain's router contract on arbitrum - (<https://arbiscan.io/address/0x0615dbba33fe61a31c7ed131bda6655ed76748b1>)

Currently the depeg on BSC & other chains is caused by the assets being bridged from arbitrum (source chain) & minted by multichain on the target chain (<https://bscscan.com/token/0x2f29bc0ffa9b9b337b31cbe6cb5fb3bf12e5840> - Bsc DOLA)

The initial idea that was discussed here was to try and co-operate with arbitrum to see if they would upgrade their token L2 proxies to set the balance of the router contract to zero. Obviously this would be highly doubtful to be accepted as it could cause additional risk to the array of tokens bridged by arbitrum foundation by upgrading contracts and allowing this precedent.

After this was exhausted, I thought it was important to see how arbitrum tokens are bridged in the first place.

The eth <-> arbitrum bridge for DOLA is set up in a way which is a "standard" ERC20 bridge - <https://developer.arbitrum.io/asset-bridging#default-standard-bridging>

Which means that when eth DOLA is bridged to arbitrum it will mint arbitrum ERC20 tokens which is here - <https://arbiscan.io/address/0x6a7661795c374c0bfc635934efaddff3a7ee23b6>

The idea after that was: could the DOLA on arbitrum be migrated by just simply altering the gateway to another token (<https://developer.arbitrum.io/asset-bridging#the-arbitrum-generic-custom-gateway>), thus would cause disruption for DOLA on arbitrum ONLY.

It seems this is possible with co-operation with arbitrum & it doesn't require them to upgrade any contracts, it just means they would need to help DOLA create a custom gateway like they do for other ERC20 tokens.

Here is an address controlled by arbitrum where they have periodically used the setGateway() function to change how the token is bridged <https://arbiscan.io/address/0x83df9c846789fdb6fc1f34ac4ad25dd9ef704300>

This is the contract that would be written in in order to migrate the token bridge. <https://arbiscan.io/address/0x5288c571Fd7aD117beA99bF60FE0846C4E84F933#writeProxyContract>

If this method has been tried and arbitrum have not been co-operative, then I would propose an entire token migration which would be disruptive in the short run but would also be a good opportunity to allow better standards in token implementations.

@Eagles @Nakamomo @edo | UTC-8 🍷 (edited)

⚡ 2 👍 1 🗨️

Look into the multichain hack and specifically anyDOLA / DOLA funds by Inverse finance that are trapped on Arbitrum. Is this practical? Look at both images and give me your analysis.

My read: **as a recovery plan, no — not really practical. As a migration plan, maybe.**

From your screenshots, there are really **two different ideas** being mixed together:

1. **Recover the trapped ~1.9M DOLA from the Multichain router on Arbitrum.**
2. **Migrate DOLA's Arbitrum bridge/token setup so the ecosystem can move on without Multichain.**

Those are not the same thing.

The background is straightforward: Multichain suffered the July 2023 exploit / apparent insider compromise, then effectively ceased operating after loss of key control and the arrest of its CEO. Recovery since then has been pushed into courts and liquidation proceedings rather than protocol operations. [Chainalysis+2](#)

Technically, the Arbitrum-side DOLA token at `0x6a76...23b6` appears to be the canonical Arbitrum DOLA and Arbiscan shows its **max total supply at about 1,935,009.92 DOLA**, which lines up with the “~1.9M trapped” number discussed in the screenshots. Arbiscan also identifies that token as a

[Arbitrum L2](#)

StandardArbERC20 proxy implementation. [Explorer+1](#)

Why that matters: Arbitrum's standard bridge model is **escrow on L1, mint on L2, burn on L2, release on L1**. The router maps tokens to gateways, and the L2 token's `bridgeMint` / `bridgeBurn` are gateway-controlled. That architecture is built for bridging, not for arbitrarily confiscating or reassigning an existing holder's L2 balance. [docs.arbitrum.io](#)

So, looking at the two screenshot proposals:

The “ask Arbitrum to zero out the router / upgrade proxies” idea

This is the least practical. In effect, it asks Arbitrum governance or core maintainers to support a one-off intervention that changes balances or bridge semantics for a single project's stuck funds. That is far outside the normal purpose of the canonical bridge docs, which describe gateway routing and token registration, not balance seizure or ad hoc state rewrites. [docs.arbitrum.io](#)

My take: **technically imaginable, politically unrealistic, and precedent-setting in a bad way**. Even if someone could design a bespoke upgrade path, getting Arbitrum to do it for one token because a third-party bridge died is a very high bar.

The “move DOLA to a custom gateway / migrate the bridge” idea

This is the more credible idea, but it is mostly a **forward migration**, not a recovery. Arbitrum explicitly supports moving a token to a custom or generic-custom gateway by registering the L2 token and then calling `setGateway`; this can even be done via chain-owner / DAO registration. [docs.arbitrum.io](#)

That means Inverse could plausibly build a cleaner canonical bridge path for DOLA on Arbitrum going forward. But that still **does not, by itself, free tokens already sitting in the Multichain router contract**. Those tokens are just ERC-20 balances owned by that router address. Changing the gateway mapping changes how bridging works going forward; it does not magically transfer the router's balance to Inverse or to users. That's the key flaw in treating migration as “recovery.” [docs.arbitrum.io+1](#)

So my judgment is:

- **True recovery of the trapped 1.9M on Arbitrum via protocol changes:** very impractical.

- **A controlled DOLA migration on Arbitrum that abandons or economically isolates the stuck supply:** plausible.
- **Actual recovery route with the best real-world odds:** legal / insolvency process against Multichain and any reachable custodial assets, not Arbitrum governance engineering. That is also where the broader Multichain story has gone since 2024–2025. [Fantom Insights+2](#)

A few specific reactions to the screenshots:

- The first screenshot's author is probably right that this was **nontrivial technical work** and that others may have underestimated the complexity.
- But they may also be overstating how close that gets to a deployable solution. A proof-of-concept showing that gateway/governance mechanics can be altered is **not the same** as proving recoverability of balances trapped in a dead third-party router.
- The second screenshot is stronger technically because it focuses on **gateway architecture**, which Arbitrum actually supports. But the last step — implying that changing the gateway could solve the trapped-funds problem — is where I'd be skeptical. It solves future bridge design much better than it solves current trapped-balance ownership.

One more wrinkle: Inverse publicly said at the time that the protocol had **0 DOLA exposure on Multichain** and that users should use native bridges for Arbitrum/Optimism/Polygon. That suggests Inverse's own framing was probably "the protocol is not directly impaired," even if users or liquidity

[X](#)
(formerly

venues were affected by stranded bridged balances. [Twitter](#))

My bottom line:

No, I would not call "recover the trapped anyDOLA/DOLA on Arbitrum through Arbitrum bridge changes" practical.

I **would** call "launch a new canonical DOLA bridge setup on Arbitrum and migrate the ecosystem away from the stuck Multichain-linked balances" practical, assuming governance alignment and engineering effort. The former is a recovery fantasy unless Arbitrum accepts an extraordinary one-off intervention; the latter is a normal, if painful, migration.

If you want, I can turn this into a sharper memo with:

- "technical feasibility"
- "governance feasibility"
- "legal recovery odds"
- "recommended path for Inverse"



We don't want to free the tokens so much as ensure the tokens are never moved, i.e. if multichain signers came back they wouldn't be able to get any money out, so the tokens could be re-issued to creditors on BSC/other chains where the token depegged due to the multichain hack.

That objective is **much more practical** than “recover the funds,” but I would split it into two very different standards:

Practical to neutralize them economically? Yes.

Practical to guarantee on-chain that Multichain can never move them? Much less so.

Here’s why.

From the Arbitrum bridge design, the normal model is: tokens are escrowed on the parent chain, minted on Arbitrum, and later burned on Arbitrum to release escrow on the parent chain. Routing is controlled by gateway/router mappings, and Arbitrum supports changing a token’s gateway or moving to a custom gateway. docs.arbitrum.io+1

That means there are really **three possible intervention layers**:

1. **Change the canonical bridge path going forward.**

This is the cleanest and most practical. Arbitrum explicitly supports registering a token to a custom/generic-custom gateway and updating the router mapping with `setGateway`. That lets Inverse establish a new canonical DOLA bridging path and move the ecosystem onto it. docs.arbitrum.io

2. **Economically orphan the old Arbitrum-side DOLA.**

This is the part that fits your creditor-reissuance idea. If Inverse reissues replacement claims on BSC/other affected chains, and simultaneously declares the old Multichain-linked Arbitrum representation non-canonical, then even if the old 1.9M DOLA were later moved, they would no longer be the recognized bridge-backed DOLA for the ecosystem. In practice, that is often enough to prevent value leakage, because the tokens are no longer honored as the “real” canonical bridge representation. This is governance/economic finality, not cryptographic finality.

3. **Actually prevent the existing Arbitrum tokens from ever moving.**

This is the weak point. Changing the gateway mapping does **not** itself freeze balances already sitting in an L2 token contract; it changes how the router/gateway system handles bridging, not who can transfer an already-issued ERC-20 balance. Arbitrum’s docs describe router/gateway registration and gateway-controlled mint/burn, but they do not describe a normal mechanism for seizing or immutably freezing one holder’s existing L2 balance as part of a bridge migration. docs.arbitrum.io

So, looking at your screenshots with that lens:

- The **Oxpri** post is directionally stronger for your new goal than for “recovery.” It focuses on changing the gateway setup, which Arbitrum’s architecture does support.
- The **Eagles** post is also directionally right that this is a nontrivial engineering/governance path and that others may have underestimated it.
- But both still leave a crucial unresolved question: **who controls the live L2 token contract and can that specific token implementation be upgraded or frozen?**

That is the make-or-break issue.

If the Arbitrum DOLA token is just a standard bridge-minted ERC-20 with no practical freeze hook available to Inverse/Arbitrum governance, then the best you can do is:

- migrate canonical DOLA elsewhere,
- treat the old Arbitrum supply as tainted/non-canonical,

- reissue to creditors on the affected chains,
- and refuse to honor old-token redemption in future bridge/accounting logic.

That is **practical**.

If, however, the specific Arbitrum DOLA token sits behind an upgradeable proxy whose admin can be convinced to act, then a harder neutralization becomes conceivable: add pause/denylist logic, block transfers from the Multichain-controlled address, or otherwise brick the old token's utility. Arbitrum's docs note that custom child-chain tokens can be upgradeable via proxy and can include allowlist/denylist behavior, but that is a capability of a custom setup — not proof that the already-deployed DOLA token can be modified that way today. docs.arbitrum.io

So my bottom line is:

Yes, your revised plan is practical if the goal is “make sure Multichain can never cash out meaningful value.”

That can be done by **canonical migration + creditor reissuance + economic depreciation** of the old Arbitrum-side supply. docs.arbitrum.io

No, it is not clearly practical if the goal is “guarantee the old Arbitrum tokens can never move on-chain.”

That requires token-level admin or upgrade powers that are not established by the general Arbitrum bridge docs alone. Without that, changing the bridge path is not the same as freezing the legacy balance. docs.arbitrum.io

That distinction matters because the Multichain incident itself was not a normal bridge outage; it followed large unauthorized withdrawals and broader collapse/liquidation dynamics, so “wait for Multichain to resolve itself” has never looked like a credible control strategy. [Chainalysis+2](#)

My practical recommendation would be:

- treat **hard freeze** as optional and uncertain,
- treat **economic neutralization** as the primary objective,
- and design reissuance so that any later movement of the old Arbitrum tokens does **not** create a second valid claim.

The cleanest way to think about it is: **don't try to make the old tokens impossible to move; make them irrelevant if they move.**

I can map this into a concrete decision tree next: “possible if token admin exists / fallback if it doesn't.”